

'tpl.php' instead of '.php'. If you want to use a different file as the view file, you may pass the filename as a second parameter to the *Run* function, like:

```
NewPost::Run('NewPost', 'newpost_alternate.tpl.php');
```

This will force QForm to use *newpost_alternate.tpl.php* as the template file. Such a technique can be used to create multiple layouts for the same page!

The view file

The view file contains the HTML skeleton of the page. The only thing you do not have to write in this file is the HTML for the individual controls, which you defined in the controller file. You would instead call the *Render* function on those controls—this makes sure that QForm takes control of their behaviour. The view file would typically begin with the HTML *head* section containing the page title, and any extra JS and CSS to be loaded. You might want to create a static PHP page to be 'included' for the purpose. Remember that, as of now, QForm does not come with any special functions to help you with the HTML *head* section. All controls must be rendered between two lines: '\$this->RenderBegin();' and '\$this->RenderEnd();'. An attempt to render a control beyond these boundaries will cause the framework to produce an error and halt. It is advisable to put the *RenderBegin()* function just after the *body* tag begins, and *RenderEnd()* just before the *body* tag closes. Again, controls will not render themselves unless you ask them to. If you removed the line '\$this->btnSubmit->Render();' from the view file, then the *submit* button will disappear from the page.

And now, the one question we left unanswered—why does *lblMessage* show the strong tags on the page instead of making the text bold? It is because QForm enables the *htmlentities* function for the text in all *QLabel* variables. In the *Form_Create()* function, just set:

```
$this->lblMessage->HtmlEntities = false;
```

You will see (Figure 2) that the *strong* tag gets back its importance!

Behaviour

The page behaviour is pretty obvious. It will accept the title and body of a post, and when the *Submit* button is hit, it creates a new post and saves it to the database, erases the contents of the title and body input boxes, and displays a message that the post was saved.

The real power of QForms

Throughout this article, we have discovered some of the core features and functionalities of QForms. Here is how they help you:

1. *Separate View and Controller files*: QForm separates the

Figure 2: With *HtmlEntities* set to false, *QLabel* honours HTML code in 'Text'

- HTML layout from behaviour. Chances of messing up HTML and PHP code are lower than ever. Since 'model' is automatically taken care of by Codegen, we are not going to worry about that.
2. *Event-driven*: You define a control, bind it to an event, and write the event-handler function for the same. QForm does the rest.
3. *AJAX is easy*: One does not need to write JS functions, remember form element names and IDs, and there is no need to deal with the *\$_POST* array either. QForm does the dirty work, allowing you to focus on 'what' to do rather than 'how' to do.
4. *AJAX to page reloads and vice versa*: Changing the behaviour of an action from a 'full page reload' to 'AJAX based' can be done by changing just one word—the action type.
5. *Less HTML than ever*: QForm produces HTML code for single elements, while you have to take care of only the overall page structure. Combined with Codegen, it means roughly 80 per cent reduction in the amount of HTML and JavaScript you would write otherwise. Also, the fact that most HTML code is automatically generated means that you have to deal with far fewer syntax errors in HTML code.
6. *Stateful*: The server knows the state of the page on the client side, and that improves security as well as ease of use for the programmer.

This article hardly scratches the surface of QForm. There are a lot of other powerful features built into QForm; some interesting ones include—control and form validation, custom controls, database-based session handling, centralised form state handling, HTML input validation and purification for preventing XSS attacks, and so on. We will peek into a few of them in the next article of this series. **END** 

By: Vaibhav Kaushal

The author is a 25 year old Web developer from Bengaluru who also happens to be a core contributor to QForm. He loves writing for technology magazines when he is not busy fiddling around with QForm or developing his website (<http://www.c-integration.com/>), where he gives QForm-related advice.